

ASSIGNMENT 8

Data-Driven Automated Testing with Selenium & Python

Student Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

1. Problem Statement

Modern web applications require robust automated testing pipelines to ensure authentication flows handle both valid and invalid user credentials seamlessly. Manual verification across multiple scenarios is repetitive and error-prone. The goal is to design a scalable, data-driven test suite that dynamically reads test credentials and expected assertion outcomes from an external file to validate web login behavior.

2. Objective

- Automate the authentication workflow for the *SauceDemo* web application using Python and Selenium WebDriver.
 - Implement Data-Driven Testing (DDT) principles by decoupling test logic from test input datasets.
 - Dynamically parse input datasets from an external CSV file.
 - Assert successful page navigation for valid accounts and ensure error elements display correctly for negative test cases.
-

3. Tools, Software, and Concepts Used

- **Programming Language:** Python 3.x
 - **Automation Tool:** Selenium WebDriver (`selenium.webdriver``)
 - **Data Handling Module:** Built-in `csv`` module (`csv.DictReader``)
 - **IDE/Editor:** Visual Studio Code
 - **Core Concepts:** Data-Driven Testing (DDT), DOM element locator strategies (By.ID, By.CSS_SELECTOR), Python control flows, conditional assertions.
-

4. Source Code & Data Implementation

Input Dataset File (`test_data.csv`):

```
username,password,expected
standard_user,secret_sauce,success
wrong_user,wrong_password,failure
standard_user,wrong_password,failure
wrong_user,secret_sauce,failure
```

Python Test Script (`Assignment8.py`):

```

from selenium import webdriver
from selenium.webdriver.common.by import By
import csv

with open("test_data.csv", "r") as file:
    data = csv.DictReader(file)

    for row in data:
        driver = webdriver.Chrome()
        driver.get("https://www.saucedemo.com/")
        driver.maximize_window()

        # Enter username & password
        driver.find_element(By.ID, "user-name").send_keys(row["username"])
        driver.find_element(By.ID, "password").send_keys(row["password"])
        driver.find_element(By.ID, "login-button").click()

        if row["expected"] == "success":
            assert "/inventory.html" in driver.current_url
            print("PASS:", row["username"])
        else:
            error = driver.find_element(By.CSS_SELECTOR, "[data-test='error']")
            assert error.is_displayed()
            print("PASS - Error displayed:", row["username"])

    driver.quit()

```

5. Execution Output Log

```

PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\seleniumassignment\Assignment 2nd wk>
python -u "c:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\seleniumassignment\Assignment 2nd
wk\Assignment8.py"
PASS: standard_user
PASS - Error displayed: wrong_user
PASS - Error displayed: standard_user
PASS - Error displayed: wrong_user
PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\seleniumassignment\Assignment 2nd wk>

```

6. Result, Observation, and Conclusion

Results: All four iterations defined within the external dataset executed successfully. The valid user credentials properly reached the landing inventory page, whereas all three invalid credential combinations successfully triggered UI error elements.

Observations: Parameterizing test cases using CSV external files eliminates hardcoded values inside automated test scripts. This setup improves maintainability and scaling capabilities for larger regression test suites.

Conclusion: The implementation successfully satisfies the requirement for data-driven login verification using Python and Selenium WebDriver.